Localisation de robots mobiles en environnement intérieur : Fusion de données odométriques et trilatération BLE par filtre de Kalman

DEROCHE Florent, DEFFAYET Jolan, GONCE Loïs, YANG Kaiyue

8 avril 2026

Résumé

La localisation de robots mobiles en environnement intérieur est un défi majeur de l'industrie 4.0 étant donné que les signaux satellites (GNSS) y sont inefficaces. Cet article présente une solution à faible coût et facilement déployable répondant à ce problème industriel. Ce système repose sur la trilatération de signaux Bluetooth Low Energy (BLE) basés sur le RSSI, émis par trois ESP-32 agissant comme ancres couplées par un filtre de Kalman en temps réel aux données odométriques des roues du robot mobile. Ce filtre de Kalman est implanté sous le framework ROS2 et la communication entre l'ESP-32 du TurtleBot et l'ordinateur qui reçoit les données se fait par serveur MQTT. Nos expérimentations montrent que ce filtrage permet de corriger la dérive des roues tout en lissant les erreurs de positionnement liées au BLE. Le système atteint une précision d'approximativement 4 mètres. Pour garantir l'accès à tous, l'ensemble du code et des démarches sont disponibles en libre accès sur la page GitHub du projet.

1 Introduction

La localisation précise des robots mobiles en environnement intérieur constitue un défi fondamental en robotique. Contrairement aux environnements extérieurs, où les systèmes de navigation par satellites (GNSS) offrent une précision centimétrique, les espaces confinés rendent ces technologies inopérantes en raison de l'atténuation du signal satellite. Cette limitation exige le recours à des approches alternatives. Parmi les technologies disponibles, le Bluetooth Low Energy (BLE) présente un intérêt particulier pour la localisation intérieure. Sa faible consommation énergétique, son déploiement simplifié et sa présence généralisée dans les équipements embarqués en font un candidat idéal pour les systèmes de localisation à coût réduit. Cependant, la mesure de la puissance du signal reçu (RSSI) souffre d'une incertitude importante due aux réflexions et interférences électromagnétiques aux environnements intérieurs, se traduisant par une estimation bruitée et instable.

Pour corriger ces limitations, la fusion de sources d'information complémentaires s'impose comme une approche robuste. L'odométrie des roues, disponible nativement sur la plupart des robots mobiles, offre précisément la complémentarité nécessaire : précise localement et à haute fréquence, elle souffre néanmoins d'une dérive cumulative qui la rend inexacte à long terme. La fusion de ces deux modalités dans un filtre de Kalman permet alors d'exploiter leurs forces respectives. En effet, la correction absolue du BLE compense la dérive odométrique, tandis que la prédiction odométrique lisse les discontinuités de la mesure BLE. Cet article présente l'implémentation et l'évaluation d'un tel système sur un robot TurtleBot Kobuki, en s'appuyant sur une infrastructure matérielle à base d'ESP32 et une architecture logicielle ROS2, dont l'ensemble du code source est mis à disposition en libre accès sur le dépôt GitHub du projet.

La suite de l'article est organisée comme suit : la section 2 détaille l'architecture matérielle et logicielle ainsi que les choix méthodologiques ; la section 3 décrit le protocole expérimental ; la section 4 présente et discute les résultats ; la section 5 conclut cette expérience.

2 Architecture du Système et Méthodologie

2.1 Choix des technologies et outils

L'architecture retenue repose sur une sélection de technologies standard dans le domaine de la robotique embarquée et de l'IoT, privilégiant la disponibilité, le faible coût et l'intégration native.

- Bluetooth Low Energy (BLE) : Protocole de communication sans fil à faible consommation énergétique, utilisé comme outil de mesure de base. Les balises BLE diffusent périodiquement un signal dont la puissance reçue est exploitée pour estimer la position du robot et de chaque ancre.
- ESP32 : Microcontrôleur Espressif intégrant nativement le BLE et le Wi-Fi sur la même puce. Il est utilisé à la fois pour les ancres fixes qui diffusent un signal BLE à puissance configurable et pour le module fixé sur le robot qui scanne les signaux des ancres. Il applique un filtre EMA sur les puissances reçues (RSSI) et transmet les mesures au broker MQTT via Wi-Fi. Son faible coût en fait le composant principal de l'infrastructure matérielle.
- MQTT Mosquitto : Protocole de messagerie léger basé sur TCP/IP, conçu pour les réseaux peu fiables ou les réseaux dont les ressources en bande passante sont limitées. Il est principalement utilisé pour les communications de machine à machine (M2M) ou les connexions de type Internet des objets (IoT). Le robot publie les RSSI filtrés des trois ancres sous forme de message JSON. Le broker Mosquitto déployé sur une machine connectée au réseau IoT IMT Nord Europe, relaie ces messages vers le nœud ROS2 de trilatération.
- ROS2 : Framework robotique open-source constituant la couche de traitement haut niveau. Son architecture modulaire en nœuds communicants via des topics, son système de transformation de repères TF2 et son outil de visualisation RViz2 facilitent l'intégration des différentes sources de données. Deux nœuds Python sont développés : trilateration_node pour le calcul de position BLE, et fusion_node pour la fusion Kalman avec l'odométrie du robot Kobuki.
- Filtre de Kalman : Algorithme d'estimation récursive combinant une prédiction et une correction dynamique. Il est utilisé pour fusionner les deux modalités de localisation à des fréquences différentes : la prédiction odométrique assure la continuité de l'estimation, tandis que la correction BLE compense la dérive cumulative de l'odométrie. Le filtre est paramétré par une matrice de bruit de processus $\mathbf{Q}$ reflétant la confiance en l'odométrie et une matrice de bruit de mesure $\mathbf{R}$ reflétant l'incertitude des estimations BLE.

2.2 Architecture matérielle et logicielle

L'architecture de notre système de localisation est principalement constituée d'une partie hardware et d'un firmware embarqué sur celui-ci, ainsi que d'une partie logicielle grâce à nos scripts ROS python et un serveur MQTT, tournant sur un PC hébergeur. Pour ce qui est de la partie matérielle, nous disposons de 4 ESP32 avec antenne intégrée, de 3 batteries portatives, d'un robot Kobuki embarquant une Raspberry-Pi et différents capteurs (Odométrie, Caméra infra-rouge...), et enfin d'un pc hébergeur du Broker MQTT sous Ubuntu avec ROS2 d'installé.

- Dans un premier temps, 3 ESP32 sont dédiés à fonctionner comme "Ancres" statiques alimentées par batterie portatives, émettrices de de signaux BLE à des emplacements bien connus et mesurés dans l'environnement où évolue le robot.
- Dans un second temps, un autre ESP32 est installé et immobilisé sur le capot du robot Kobuki, et est destiné à fonctionner comme "station" de mesure de la puissance du signal émit par chaque Ancres Bluetooth, et de communiquer les résultats au serveur MQTT par wifi.
- Enfin dans un dernier temps, les résultats des mesures RSSI sont reçus par le serveur MQTT et sont ensuite transmis à nos packages python ROS2, qui effectuent les calculs de trilatération, de fusion de données, ainsi que de la collecte des données odométriques des roues du Kobuki

Au niveau des logiciels embarqués, les firmwares en C/C++ des ESP32 de type "Ancres" et "Robot", ont été programmés et flashés sur ces micro-contrôleurs grâce à l'environnement de développement Arduino fourni par l'outil "PlatformIO" et l'IDE VS Code. Les principales librairies utilisées sont la librairie standard pour le BLE, ainsi que PubSubClient et ArduinoJson pour l'envoi de messages JSON au serveur MQTT. De ce fait, pour faire le lien entre les ESP, le robot Kobuki et notre PC Ubuntu pour les scripts ROS, nous avons utilisées le broker Mosquitto. Un point important est que tous les appareils doivent être connectés sur le même réseau wifi avec la bonne

adresse IP de configurée afin que la communication s'effectue correctement.

Au niveau du package ROS, nous avons un launch file pour lancer nos 3 noeuds principaux : "ble localization", "fusion node" et "ble origin broadcaster".

— "ble localisation" récupère les données RSSI via les messages Json du broker MQTT, et effectue la trilatération bluetooth en résolvant l'équation de la position via la méthode des moindres carrés :

$$d_i = \sqrt{(x - x_i)^2 + (y - y_i)^2 + (z - z_i)^2} \tag{1}$$

avec x, y les coordonnées à calculer de notre robot, z la hauteur connue de l'ESP32 du robot et x_i, y_i et z_i les coordonnées de nos ancres.

— "fusion node" récupère cette première estimation de la position uniquement par bluetooth, et fusionne ces données avec l'odométrie des roues du robot Kobuki à l'aide d'un filtre de Kalman en 2 dimensions afin de lisser nos résultats.

— "ble origin broadcaster" publie simplement un repère statique dans notre carte, aux coordonnées de notre point de référence réel (Origine), à l'emplacement de départ du robot.

2.3 Modélisation du signal BLE

Le modèle de perte de propagation logarithmique (*Log-Distance Path Loss Model*), qui établit une relation entre la puissance du signal BLE reçu et la distance entre l'émetteur et le récepteur, est utilisé pour estimer la distance qui sépare chaque ancre du robot.

$$RSSI(d) = RSSI(d_0) - 10n \log_{10}\left(\frac{d}{d_0}\right) + X_\sigma \tag{2}$$

Où :

$RSSI(d)$ Puissance du signal reçu à la distance d ;

$RSSI(d_0)$ Puissance de référence mesurée à une distance d_0 connue ;

n Exposant de perte de trajet ;

X_σ Bruit.

2.4 Fusion de données par Filtre de Kalman

Afin d'estimer de manière fiable et continue la position du robot Kobuki, nous avons implémenté une architecture de fusion de données reposant sur un filtre de Kalman en 2 dimensions. L'objectif est de lisser la position par bluetooth obtenue toutes les 500 ms, avec l'odométrie des roues, qui est récupérée à très haute fréquence avec une bonne précision. L'odométrie du robot constitue la base de notre étape de prédiction : elle est efficace pour les déplacements fluides à court terme, mais souffre d'une dérive cumulative au fil du temps.
À l'inverse, notre système de trilatération Bluetooth agit comme étape de correction en fournissant des coordonnées absolues dans le repère de la pièce "ble origin". Toutefois, le signal radio étant par nature instable, ces mesures brutes présentent un bruit important.

— **Innovation (Erreur de mesure) :**

$$Y_k = Z_k - \hat{X}_k^-$$

— **Covariance de l'innovation :**

$$S_k = P_k^- + R$$

— **Gain de Kalman :**

$$K_k = P_k^- S_k^{-1}$$

— **Mise à jour de l'état (Position fusionnée) :**

$$\hat{X}_k = \hat{X}_k^- + K_k Y_k$$

— **Mise à jour de la matrice de covariance :**

$$P_k = (I - K_k) P_k^-$$

L'enjeu majeur réside dans la calibration du filtre avec un choix de valeurs cohérentes pour les matrices de covariances Q et R, pour éviter de gros sauts dans l'estimation de la position en temps réel. (Ici Q est minimale, et R très élevée)

2.5 Récupération et analyse des données

Afin d'évaluer les performances des méthodes employées, il faut pouvoir les comparer de manière synchrone avec les positions exactes générées par le SLAM. Pour ce faire, un script d'acquisition ainsi qu'un script d'analyse ont été mis en place.
L'enregistrement des données est assuré par un nœud ROS2 dédié développé en Python. Ce nœud s'abonne à la fois aux topics de positionnement brut (trilatération BLE) et de positionnement filtré (filtre de Kalman). Parallèlement, la position exacte du robot est extraite depuis l'arbre des transformations de ROS2. Ensuite, le script projette les positions dans un repère commun. Enfin, les positions sont exportées vers un fichier CSV.
Un deuxième script analyse les données extraites et calcule l'erreur d'estimation moyenne (MAE), l'erreur quadratique moyenne (RMSE), calculée avec la formule ci-dessous, ainsi que le *jitter* moyen qui permet de quantifier le tremblement de la position estimée d'un moment au suivant.

$$RMSE = \sqrt{\frac{1}{n}\sum_{i=1}^{n}(y_i - \hat{y}_i)^2} \tag{3}$$

3 Protocole Expérimental

3.1 Description de l'environnement expérimental

Afin d'évaluer les performances du système de localisation, une expérimentation a été menée dans un environnement intérieur. La zone de test est une pièce d'une superficie de 50m^2 (10m par 5m). Cet environnement comprend divers obstacles tels que des obstacles au sol entravant la mobilité du robot et des murs en béton empêchant la réception de signaux GNSS.

3.2 Méthode de référence de localisation

Pour évaluer la précision et la fiabilité du système de localisation, il faut comparer les résultats obtenus à une position estimée fiable de référence. Pour obtenir ces positions fiables, un système basé sur la technologie LiDAR couplée à une approche Simultaneous Localization And Mapping (SLAM) est utilisé. Le LiDAR dont le robot est équipé génère directement des données géométriques d'une grande précision. Lors des déplacements, le capteur scanne les obtsacles et l'algorithme de SLAM construit une carte de l'environnement en temps réel tout en calculant la position exacte du robot au sein de cette carte. Nous obtenons ainsi un positionnement de référence avec une précision centimétrique.

3.3 Méthodologie et déroulé de l'expérience

Cette section détaille la mise en œuvre du système de localisation. Dans le but d'assurer la reproductibilité des résultats, un protocole expérimental a été défini. Celui-ci s'articule sur la préparation de la zone d'expérimentation suivie d'une phase d'acquisition des données durant laquelle le robot se déplace.

Pour cela, voici la liste du matériel nécessaire :
- 4 ESP32 DevKit C-V4
- 3 batteries externes
- 4 câbles USB-C vers micro USB
- 1 turtlebot Kobuki équipé de la technologie LiDAR
- 1 ordinateur
- 1 mètre
- 1 manette

Voici les étapes à suivre :

1. initialiser le serveur MQTT
2. flasher 3 ESP32 qui feront office d'ancres avec le code approprié
3. flasher l'ESP32 qui ira sur le robot
4. brancher 3 ESP32 servant d'ancres à leurs batteries externes

5. définir un point d'origine dans la pièce servant de point de départ pour le robot
6. définir une orientation d'origine pour le robot
7. placer les ancres dans l'environnement
8. relever les coordonnées des ancres
9. brancher une ESP32 au robot
10. placer le robot au point d'origine dans l'orientation définie précédemment
11. allumer le robot et les ancres
12. mesurer le RSSI des ancres à 1m et renseigner la valeur obtenue dans le code
13. lancer l'algorithme de SLAM
14. lancer les nœuds ROS2 d'estimation de la position (BLE seul et filtre de Kalman) et de récupération des données
15. déplacer le robot pour avoir des données pertinentes

4 Résultats et Discussion

4.1 Analyse des résultats

Les résultats suivants sont obtenus :

Méthode d'estimation	**MAE (m)**	**RMSE (m)**	**Jitter moyen (m)**
Trilatération BLE brute	3,929	4,064	0,235
Filtre de Kalman	3,785	3,921	0,044

Table 1 – Comparaison des erreurs de positionnement par rapport à la position exacte (SLAM).

On voit que le filtre de Kalman donne une estimation plus précise, une amélioration de 3,6% est obtenue avec le filtre de Kalman sur la MAE et une amélioration de 3,5% sur la RMSE. Ces améliorations sont minimes mais l'ajout de l'odométrie permet d'obtenir une meilleure stabilité spatiale comme on peut le voir avec les jitters moyens. On obtient en effet une amélioration de 81,3 % du jitter moyen.

4.2 Limites et pistes d'amélioration du système

Bien que le filtre de Kalman améliore le résultat, une erreur d'estimation de 4 mètres reste élevée et cela est inhérent à la technologie employée. Le RSSI est très sensible aux réflexions sur les murs et obstacles ce qui rend l'estimation très instable même dans le cas où le robot est immobile.

En piste d'amélioration, rajouter des ancres permettrait d'améliorer la précision car trois ancres est le minimum nécessaire pour faire une trilatération mais en cas d'obstruction d'une de ces ancres, les calculs géométriques seront faussés. Utiliser une autre méthode de localisation comme du "fingerprinting" avec réseau de neurones serait aussi une piste d'amélioration envisageable. Il serait possible de mesurer le Bluetooth à de nombreux points dans toute la salle,ainsi que les positions exactes grâce au SLAM, puis d'en faire des jeux de données sur lesquels entraîner le réseau de neurone, qui ainsi trouverait des estimations de la position en "apprenant" la réflexion du signal Bluetooth.

5 Conclusion

Ce travail a présenté un système complet de localisation intérieure sans infrastructure GNSS, combinant trilatération BLE et odométrie des roues dans un filtre de Kalman 2D, déployé sur un robot Kobuki via le framework ROS2. L'architecture proposée, basée sur des composants à faible coût (ESP32, BLE, MQTT), est fonctionnelle et démontre la faisabilité technique de l'approche dans un environnement réel.

Les résultats expérimentaux quantifient précisément les apports et les limites du système. La trilatération BLE seule atteint une MAE de 3,929 m, un RMSE de 4,064 m et un jitter moyen de 0,235 m. L'ajout du filtre de Kalman réduit ces valeurs à respectivement 3,785 m, 3,921 m et 0,044 m. la réduction du jitter de 81,3% constitue le résultat le plus significatif de cette étude. Elle

démontre que la fusion odométrique apporte une stabilité spatiale : les discontinuités et instabilités de la trilatération BLE brute sont quasiment absorbées par la prédiction odométrique, produisant une trajectoire estimée plus stable et régulière.

Finalement, ce travail confirme que la fusion de la trilatération BLE et de l'odométrie par filtre de Kalman constitue une approche accessible et déployable à faible coût pour la localisation intérieure de robots mobiles. Si la précision absolue obtenue reste insuffisante pour une navigation autonome, les résultats valident le principe architectural et identifient les leviers d'amélioration : ajout d'ancres, calibration du modèle de propagation, et recours à des méthodes d'apprentissage par réseau de neurones.

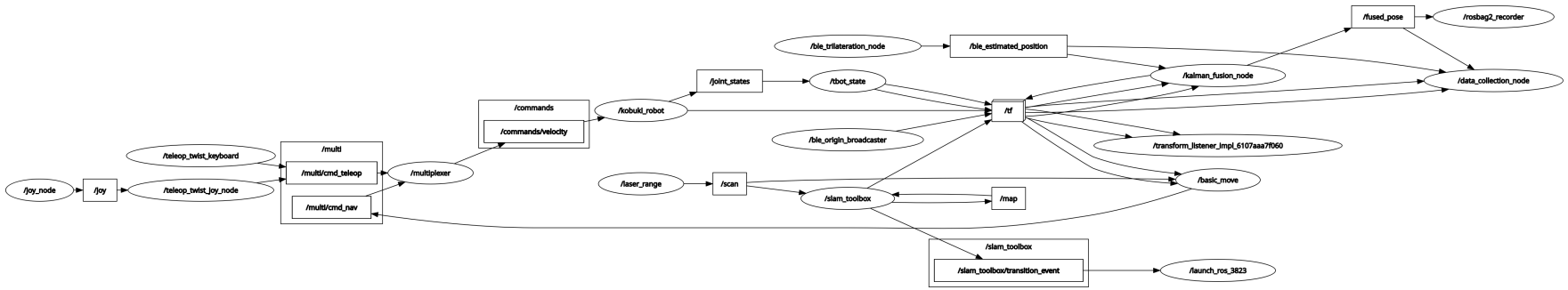

FIGURE 1 – Graphe de communication ROS (rqt_graph)